



UNIVERSIDADE FEDERAL DO CARIRI – UFCA
CURSO DE CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: LÓGICA PARA CIÊNCIA DA COMPUTAÇÃO
PROFESSOR: LUIS HENRIQUE BUSTAMANTE

Trabalho de Formalização em SAT — N-Rainhas

<https://github.com/lilyllia/trabalho-sat-rainhas.git>

Emily Fernanda da Silva Galvão Modesto
Lara Gardênia Alves Rodrigues
Yasmim Monteiro Gomes

Juazeiro do Norte, 24 de agosto de 2026

SUMÁRIO

1. Introdução.....	3
1.1 Descrição informal do problema.....	3
1.2 Por que N-Rainhas é interessante do ponto de vista computacional.....	3
2. Formalização Matemática.....	4
2.1 Variáveis proposicionais e codificação.....	4
2.2 Famílias de cláusulas em CNF.....	4
2.2.1 Ao menos uma por linha.....	4
2.2.2 No máximo uma por linha.....	4
2.2.3 No máximo uma por coluna.....	4
2.2.4 No máximo uma por diagonal (principal e secundária).....	4
2.3 Contagem de variáveis e cláusulas em função de N.....	5
3. Implementação do Gerador.....	6
3.1 Estrutura geral do código.....	6
3.2 Verificação de sanidade (contagem declarada vs. gerada).....	6
3.3 Trechos de código comentados.....	6
4. Experimentação.....	8
4.1 Metodologia.....	8
4.2 Tabela de resultados.....	8
4.3 Soluções reconstruídas.....	9
5. Análise e Conclusão.....	13
5.1 A formalização foi eficiente?.....	13
5.2 Formalizações alternativas possíveis.....	13
5.3 Relação entre complexidade do problema e desempenho observado.....	13
5.4 Comparação com outras abordagens de resolução.....	13
5.4.1. Backtracking.....	13
5.4.2. Forward checking / AC-3.....	14
5.4.3. Busca local (min-conflicts).....	14
5.4.4. Conclusão da análise.....	14

1. Introdução

1.1 Descrição informal do problema

O problema das N-Rainhas consiste em posicionar N rainhas em um tabuleiro de xadrez de tamanho $N \times N$ de modo que nenhuma rainha ataque outra, logo não pode haver duas rainhas na mesma linha, na mesma coluna ou na mesma diagonal.

1.2 Por que N-Rainhas é interessante do ponto de vista computacional

O problema das N-Rainhas é interessante porque, apesar de ser simples de entender, possui um espaço de busca que cresce muito rapidamente conforme o valor de N aumenta. Existem N^N maneiras de posicionar N rainhas em N linhas sem nenhuma restrição, mas apenas uma pequena parte dessas possibilidades representa uma solução válida.

Esse crescimento torna o problema um ótimo exemplo para comparar diferentes estratégias de busca, como o backtracking e os SAT solvers modernos. Além disso, o problema envolve restrições de linhas, colunas e diagonais, o que facilita a representação lógica. Cada uma dessas regras do xadrez pode ser transformada diretamente em um conjunto de cláusulas na forma normal conjuntiva (CNF), tornando o N-Rainhas também um bom exercício de modelagem e resolução de problemas lógicos.

2. Formalização Matemática

2.1 Variáveis proposicionais e codificação

Para representar as posições no tabuleiro $N \times N$, a definição das variáveis é feita da seguinte maneira: Definimos X_{ij} = "há uma rainha na linha i , coluna j ", onde i e $j \in [1, \dots, N]$.

Codificação linear: Como os SAT solvers operam com índices unidimensionais positivos, mapeamos a matriz bidimensional em um número inteiro único através da fórmula de codificação: $\text{var}(i,j) = N \cdot (i - 1) + j$.

2.2 Famílias de cláusulas em CNF

As restrições do problema das N-Rainhas foram divididas em famílias nomeadas. Para cada uma, detalhamos a cláusula genérica correspondente e a justificativa de necessidade e suficiência.

2.2.1 Ao menos uma por linha

Cláusula genérica: Para cada linha $i \in [1, \dots, N] : (X_{i1} \vee X_{i2} \vee \dots \vee X_{iN})$.

Justificativa: Esta restrição é necessária para garantir que nenhuma linha do tabuleiro fique vazia. Sem ela, um tabuleiro sem nenhuma rainha seria considerado válido pelo solver.

2.2.2 No máximo uma por linha

Cláusula genérica: Para cada linha $i \in [1, \dots, N]$ e para todo par de colunas $j < k$: $(\neg X_{ij} \vee \neg X_{ik})$.

Justificativa: Previne que duas ou mais rainhas dividam a mesma linha. Quando combinada à regra de "ao menos uma por linha", torna-se suficiente para garantir que exista exatamente uma rainha ocupando cada linha, garantindo o total de N rainhas no tabuleiro final.

2.2.3 No máximo uma por coluna

Cláusula genérica: Para cada coluna $j \in [1, \dots, N]$ e para todo par de linhas $i < k$: $(\neg X_{ij} \vee \neg X_{kj})$

Justificativa: Captura a restrição vertical do xadrez, sendo necessária para assegurar que duas rainhas nunca se ataquem através de uma mesma coluna.

2.2.4 No máximo uma por diagonal (principal e secundária)

Cláusula genérica: Agrupando as variáveis pela constante de diagonal descendente $d = i - j$. Para cada grupo d , consideram-se todos os pares de coordenadas distintas pertencentes a este grupo: $(\neg X_{ij} \vee \neg X_{kl})$.

Justificativa: Diagonais principais ($\searrow$) são caracterizadas pela mesma diferença entre o índice da linha e da coluna. Esta família de cláusulas impede que as rainhas ataquem umas às outras nesses eixos inclinados.

Cláusula genérica: Agrupando as variáveis pela constante de diagonal ascendente $d = i + j$. Para cada grupo d , consideram-se todos os pares distintos pertencentes a este grupo: $(\neg X_{ij} \vee \neg X_{kl})$.

Justificativa: Diagonais secundárias ($\nearrow$) possuem a mesma soma dos índices de linha e coluna. Esta cláusula é necessária para bloquear ataques diagonais ascendentes, complementando todas as direções de ataque válidas no xadrez.

2.3 Contagem de variáveis e cláusulas em função de N

Total de Variáveis: O tabuleiro possui N linhas e N colunas, resultando em N^2 variáveis.

Cláusulas "ao menos uma por linha": Uma única cláusula restritiva para cada linha, resultando em N cláusulas.

Cláusulas "no máximo uma por linha": Para cada uma das N linhas, formamos pares entre as N colunas disponíveis, totalizando: $N \cdot C(N,2) = N \cdot N \cdot (N - 1) / 2$ cláusulas.

Cláusulas "no máximo uma por coluna": Utilizando a simetria com as linhas, para cada uma das N colunas avaliamos os pares entre as linhas: $N \cdot C(N,2) = N \cdot N \cdot (N - 1) / 2$ cláusulas.

Cláusulas de diagonais: Em um tabuleiro $N \times N$, existem $2N - 1$ diagonais em cada direção (principais e secundárias). Para uma diagonal específica de comprimento L (onde L varia de 1 até N e retrocede a 1), existem $C(L,2)$ pares de "no máximo um". A contagem total será a soma de $C(L,2)$ computada sobre todas as diagonais.

3. Implementação do Gerador

3.1 Estrutura geral do código

O código do gerador e do validador de soluções foi implementado em Python, utilizando uma estrutura modular focada na execução via linha de comando. O projeto é composto por dois scripts principais:

gerador.py: É o script responsável por receber o tamanho N da instância como argumento, aplicar a função de codificação linear, gerar as famílias de cláusulas proposicionais armazenando-as em uma lista, realizar a validação matemática da contagem e, finalmente, imprimir o resultado no formato universal DIMACS CNF diretamente na saída padrão (stdout).

visualizador.py: Atua como um decodificador do modelo-solução. Ele lê o arquivo de texto de saída gerado pelo SAT solver e reconstrói visualmente o tabuleiro no terminal, posicionando as rainhas ('Q'). Este script foi projetado para suportar nativamente tanto o formato de saída do MiniSat quanto o formato de competições (utilizando prefixos 's' e 'v') comum aos solvers Kissat e CaDiCaL.

3.2 Verificação de sanidade (contagem declarada vs. gerada)

A verificação de sanidade é uma etapa de blindagem crítica do código, implementada na função `checar_sanidade`. Esta etapa garante que a contagem de variáveis e cláusulas declarada no cabeçalho do formato DIMACS corresponda exatamente às restrições geradas para que o solver não rejeite o arquivo. Para isso, o código calcula analiticamente o total esperado de cláusulas, somando as N^2 cláusulas mínimas por linha com as fórmulas $N \times (N \times (N - 1)) / \div 2$ de máximo para linhas e colunas. Em seguida, adiciona a contagem das diagonais, que é obtida pelo somatório de combinações $C(L, 2)$ para os triângulos do tabuleiro e a diagonal central. Caso o número final de cláusulas geradas no array não bata com o somatório dessas equações matemáticas, o script envia um erro para a saída de erro padrão (sys.stderr) e aborta a execução (sys.exit(1)), garantindo que a saída limpa (stdout) não seja corrompida. A eficácia deste método é confirmada nos testes iniciais, onde a estrutura previu e gerou com precisão cirúrgica 80 cláusulas para $N=4$, 296 cláusulas para $N=6$ e 736 cláusulas para $N=8$.

3.3 Trechos de código comentados

Para mostrar as principais escolhas de engenharia, estão abaixo alguns trechos do código-fonte que são responsáveis pela modelagem das lógicas principais. Esses trechos mostram como as decisões foram colocadas em prática.

1. Função de Mapeamento (Codificação linear): Como a Lógica Proposicional aplicada a SAT solvers não compreende variáveis bidimensionais, a função abaixo foi utilizada no gerador.py para transformar um par de coordenadas do tabuleiro em um único identificador inteiro:

```
def var(i, j, n):  
    """Codificação linear: linha i, coluna j -> variável 1..n^2"""  
    return n * (i - 1) + j
```

2. Família de cláusulas das Diagonais: Pra iterar de maneira concisa sobre as diagonais principais do tabuleiro (onde a diferença $i - j$ entre as coordenadas forma uma constante d), a implementação

constrói agrupamentos por linha de inclinação e então exige que, no máximo, apenas uma variável dentro daquele array temporário seja verdadeira (combinação dois a dois):

```
# d varia de (1 - n) até (n - 1)
for d in range(1 - n, n):
    diagonal = []
    for i in range(1, n + 1):
        j = i - d
        if 1 <= j <= n:
            diagonal.append(var(i, j, n))

    # combina dois a dois dentro da diagonal
    for p in range(len(diagonal)):
        for q in range(p + 1, len(diagonal)):
            clausulas.append([-diagonal[p], -diagonal[q]])
```

3. Compatibilidade Universal de Saída: Pra permitir a troca de SAT solvers de acordo com os requisitos de experimentação, o visualizador.py ignora as linhas de comentários e checa simultaneamente pelos padrões do MiniSat e do formato competitivo SAT Competition antes de parsear as literais:

```
# verifica se é SAT (suporta formato MiniSat e Formato Competição)
if linha == "SAT" or linha == "s SATISFIABLE":
    is_sat = True
elif linha == "UNSAT" or linha == "s UNSATISFIABLE":
    print("O resultado é UNSAT (Insatisfável).")
    return

# se SAT, começa a coletar as variáveis
elif is_sat:
    # remove o prefixo 'v ' usado no kissat/cadical
    if linha.startswith('v '):
        linha = linha[2:]
```

4. Experimentação

A experimentação foi feita abordando os aspectos práticos da modelagem do problema das N -Rainhas, detalhando o ambiente de execução, os resultados numéricos obtidos pelo solver e a representação visual das soluções geradas para tabuleiros de tamanho $N \in \{4, 6, 8, 10, 12, 14\}$.

4.1 Metodologia

Para validar experimentalmente a formalização matemática proposta, foi desenvolvido um fluxo de testes que integra a geração automática das fórmulas lógicas com o processamento de um SAT solver.

- **Solver utilizado:** para a parte de experimentação foi utilizado o **Kissat SAT Solver** (versão 4.0.4)
- **Ambiente de desenvolvimento:** os experimentos foram executados de forma nativa no ambiente Linux, utilizando o WSL2 (Windows Subsystem for Linux) sob uma distribuição Ubuntu (22.04 LTS); as execuções foram realizadas em um computador com um processador Intel Core i3 e 4GB de memória RAM
- **Fluxo e comandos de execução:** para cada tamanho de instância N , a execução seguiu os seguintes passos:
 1. **Geração do arquivo DIMACS CNF:** o `gerador.py` recebe o tamanho da instância N e direciona o arquivo gerado via saída padrão para o diretório de experimentos:
`python3 gerador/gerador.py 8 > experimentos/rainhas_8.cnf`
 2. **Execução do SAT Solver:** o solver kissat lê a fórmula lógica do arquivo `.cnf` gerado e salva o log completo da resolução (incluindo variáveis valoradas, estatísticas e tempo de processamento) em um arquivo `.txt` na pasta de experimentos:
`kissat experimentos/rainhas_8.cnf > experimentos/saida_8.txt`
 3. **Visualização e Reconstrução:** o `visualizador.py` lê a saída correspondente do solver e renderiza graficamente o tabuleiro final no terminal para conferência:
`python3 gerador/visualizador.py experimentos/saida_8.txt 8`

4.2 Tabela de resultados

Os dados experimentais coletados dos arquivos de log gerados pelo Kissat estão apresentados na tabela abaixo.

N	Variáveis	Cláusulas	Resultado	Tempo_CPU_segundos
4	16	80	SAT	0.01
6	36	296	SAT	0.05
8	64	736	SAT	0.03
10	100	1480	SAT	0.06
12	144	2608	SAT	0.14
14	196	4200	SAT	0.14

Para a menor instância ($N = 4$), o somatório das famílias de restrições de linha, coluna e diagonais resultou rigorosamente em **80 cláusulas** (4 (mínimo por linha) + 24 (linhas) + 24 (colunas) + 28 (diagonais)). Para a maior instância testada ($N = 14$), a complexidade de restrições escalou gerando exatamente **4200 cláusulas**.

4.3 Soluções reconstruídas

Para cada uma das instâncias testadas, o visualizador decodificou as variáveis proposicionais positivas encontradas pelo solver e gerou as posições das rainhas. A seguir, é apresentado a reconstrução visual e as coordenadas de posicionamento para cada tamanho de tabuleiro.

Nas visualizações abaixo, (Q) indica a posição de uma rainha e (.) indica uma casa vazia.

1. Instância N = 4 (Tabuleiro 4x4)

Coordenadas das Rainhas (Linha, Coluna): (1, 2), (2, 4), (3, 1), (4, 3)

```
==== Solução encontrada para N = 4 ====  
  
  . Q . .  
  . . . Q  
  Q . . .  
  . . Q .  
  
=====
```

2. Instância N = 6 (Tabuleiro 6x6)

Coordenadas das Rainhas (Linha, Coluna): (1, 4), (2, 1), (3, 5), (4, 2), (5, 6), (6, 3)

```
==== Solução encontrada para N = 6 ====  
  
  . . . Q . .  
  Q . . . . .  
  . . . . Q .  
  . Q . . . .  
  . . . . . Q  
  . . Q . . .  
  
=====
```

3. Instância N = 8 (Tabuleiro 8x8)

Coordenadas das Rainhas (Linha, Coluna): (1, 4), (2, 7), (3, 5), (4, 2), (5, 6), (6, 1), (7, 3), (8, 8)

==== Solução encontrada para N = 8 ====

.	.	.	Q	.	.	.	.
.	.	.	.	.	Q	.	.
.	.	.	.	Q	.	.	.
.	Q	.	.	.	.	.	.
.	.	.	.	.	Q	.	.
Q	.	.	.	.	.	.	.
.	.	Q	.	.	.	.	.
.	.	.	.	.	.	.	Q

=====

4. Instância N = 10 (Tabuleiro 10x10)

Coordenadas das Rainhas (Linha, Coluna): (1, 6), (2, 3), (3, 9), (4, 7), (5, 1), (6, 4), (7, 2), (8, 5), (9, 8), (10, 10)

==== Solução encontrada para N = 10 ====

.	.	.	.	.	Q	.	.	.	.
.	.	Q	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	Q	.
.	.	.	.	.	.	Q	.	.	.
Q	.	.	.	.	.	.	.	.	.
.	.	.	Q	.	.	.	.	.	.
.	Q	.	.	.	.	.	.	.	.
.	.	.	.	Q	.	.	.	.	.
.	.	.	.	.	.	.	Q	.	.
.	.	.	.	.	.	.	.	.	Q

=====

5. Instância N = 12 (Tabuleiro 12x12)

Coordenadas das Rainhas (Linha, Coluna): (1, 7), (2, 9), (3, 4), (4, 6), (5, 1), (6, 11), (7, 2), (8, 5), (9, 3), (10, 8), (11, 10), (12, 12)

==== Solução encontrada para N = 12 ====

```
. . . . . Q . . . . .
. . . . . . . Q . . .
. . . Q . . . . . . .
. . . . . Q . . . . .
Q . . . . . . . . . .
. . . . . . . . . Q .
. Q . . . . . . . . .
. . . . . Q . . . . .
. . Q . . . . . . . .
. . . . . . . Q . . .
. . . . . . . . Q . .
. . . . . . . . . Q
```

=====

6. Instância N = 14 (Tabuleiro 14x14)

Coordenadas das Rainhas (Linha, Coluna): (1, 11), (2, 4), (3, 8), (4, 5), (5, 9), (6, 1), (7, 13), (8, 2), (9, 6), (10, 3), (11, 7), (12, 10), (13, 12), (14, 14)

==== Solução encontrada para N = 14 ====

```
. . . . . . . . . Q . . .
. . . Q . . . . . . . . .
. . . . . Q . . . . . . .
. . . . . Q . . . . . . .
Q . . . . . . . Q . . . .
. . . . . . . . . . . Q .
. Q . . . . . . . . . . .
. . . . . Q . . . . . . .
. . Q . . . . . . . . . .
. . . . . . Q . . . . . .
. . . . . . . Q . . . . .
. . . . . . . . Q . . . .
. . . . . . . . . Q . . .
```

=====

Prova Formal de Correção Manual (N = 4)

Para comprovar que a solução obtida pelo solver para N=4 é logicamente consistente com as regras do xadrez, realizamos a verificação matemática manual sobre o conjunto de coordenadas retornado pelo visualizador, representado pelos pares lineares $\{(1, 2), (2, 4), (3, 1), (4, 3)\}$:

1. **Verificação de Linhas e Colunas (Unicidade):** Como cada coordenada (i, j) possui índices de linha $i \in \{1, 2, 3, 4\}$ distintos, garante-se que cada linha contém exatamente uma rainha. O mesmo é válido para as colunas ocupadas $j \in \{2, 4, 1, 3\}$, onde a ausência de elementos repetidos certifica a restrição de no máximo uma rainha por coluna.
2. **Verificação de Diagonais Descendentes (Diferença constante $i - j$):** No xadrez, rainhas compartilham a mesma diagonal principal se a diferença entre suas coordenadas for idêntica. Calculando a diferença para cada rainha da solução:
 - Rainha 1: $1 - 2 = -1$
 - Rainha 2: $2 - 4 = -2$
 - Rainha 3: $3 - 1 = 2$
 - Rainha 4: $4 - 3 = 1$
 - Como o conjunto de diferenças $\{-1, -2, 2, 1\}$ não apresenta valores duplicados, não há conflitos de diagonal descendente.
3. **Verificação de Diagonais Ascendentes (Soma constante $i + j$):** Rainhas compartilham diagonais secundárias se a soma de suas coordenadas for idêntica. Calculando a soma para cada rainha da solução:
 - Rainha 1: $1 + 2 = 3$
 - Rainha 2: $2 + 4 = 6$
 - Rainha 3: $3 + 1 = 4$
 - Rainha 4: $4 + 3 = 7$
 - Como o conjunto de somas $\{3, 6, 4, 7\}$ não possui termos repetidos, não há conflitos de diagonal ascendente.

5. Análise e Conclusão

5.1 A formalização foi eficiente?

A formalização que usamos foi feita de forma correta e eficiente nas instâncias testadas, visto que o solver conseguiu encontrar uma solução em milissegundos mesmo para $N=8$ e que a contagem de cláusulas geradas é a mesma que foi prevista em todos os casos.

5.2 Formalizações alternativas possíveis

Uma boa alternativa para reduzir o número de cláusulas seria a codificação binária, em que cada posição seria representada por $\log_2 N$ variáveis binárias em vez de uma variável por par (linha, coluna).

Isso reduziria de forma considerável o número de variáveis, no entanto tornaria as restrições de "no máximo uma por linha/coluna/diagonal" mais complexas de expressar em CNF, exigindo cláusulas auxiliares para comparar valores binários. Como as instâncias que testamos no trabalho possuem um tamanho menor, a codificação one-hot foi suficiente e mais direta de implementar e verificar.

5.3 Relação entre complexidade do problema e desempenho observado

Embora a quantidade de cláusulas aumente de forma cúbica conforme N cresce, o tempo de execução observado permaneceu na faixa de milissegundos em todas as instâncias testadas ($N \leq 8$). Com esse resultado podemos observar como as técnicas utilizadas pelos SAT solvers modernos, como a propagação unitária e o aprendizado de cláusulas, conseguem lidar de maneira eficiente com instâncias que ainda são relativamente pequenas.

A visualização do problema como uma busca combinatória também ajuda a compreender esse comportamento. Mesmo que o espaço de busca cresça rapidamente, chegando a N^N possibilidades, o solver consegue eliminar grande parte dessas opções de maneira rápida ao utilizar as restrições presentes nas cláusulas. Dessa forma, muitas possibilidades são descartadas antes mesmo de serem totalmente exploradas.

5.4 Comparação com outras abordagens de resolução

A modelagem do problema das N -Rainhas como uma instância SAT é apenas uma das inúmeras estratégias possíveis de resolução. Comparemos a abordagem adotada com dois métodos de busca combinatória, o backtracking com poda e a propagação de restrições.

5.4.1. Backtracking

O algoritmo de backtracking soluciona o problema posicionando as rainhas linha por linha e voltando atrás sempre que uma atribuição gera conflito com rainhas que já foram posicionadas. Sua complexidade no pior caso é da ordem de $O(N!)$, mas na prática a poda dos ramos inválidos reduz consideravelmente a quantidade de possibilidades a serem analisadas, visto que é descartada uma opção no momento em que se é identificado um conflito de coluna ou diagonal.

Nos resultados experimentais que nós obtemos é possível observar que, para as instâncias testadas ($N = 4, 6$ e 8), o SAT solver encontrou solução em poucos milissegundos: 0,0022s, <0,001s e 0,00076s, respectivamente. Tais valores se assemelham aos que poderiam ser obtidos por uma implementação eficiente de backtracking para instâncias desse tamanho.

A diferença principal entre os dois métodos não se encontra na velocidade para esses casos com instâncias reduzidas, mas na forma como cada um realiza a busca. Enquanto o

backtracking elimina possibilidades quando encontra um conflito, o SAT solver utiliza técnicas como a propagação unitária e o aprendizado de cláusulas. Assim, com o SAT solver, é possível aproveitar os conflitos encontrados durante a busca para evitar que situações semelhantes sejam exploradas novamente.

5.4.2. Forward checking / AC-3

A propagação de restrições busca identificar possíveis conflitos o quanto antes. Quando uma rainha é colocada em uma posição, as opções disponíveis para as próximas variáveis são reduzidas, removendo valores que já se sabe que não poderão fazer parte de uma solução, sem a necessidade de testar de forma exaustiva.

Esse funcionamento é bastante parecido com a unit propagation utilizada pelos SAT solvers modernos, visto que os dois procuram eliminar antecipadamente caminhos que, com base nas informações já disponíveis, não podem levar a uma solução válida.

A principal diferença é que solvers CDCL, como o que foi utilizado neste trabalho, combinam a propagação com outras estratégias, como heurísticas para escolher as variáveis e o aprendizado de cláusulas. Logo, em problemas maiores ou com mais restrições, eles tendem a ser mais eficientes e robustos do que uma implementação de AC-3 utilizada de forma isolada.

5.4.3. Busca local (min-conflicts)

Para complementar a análise, também é importante mencionar a abordagem de busca local apresentada por Minton et al. (1992), que é capaz de resolver instâncias de N-Rainhas com valores de N na ordem de milhões em um tempo praticamente constante.

Isso ocorre porque o N-Rainhas, para valores grandes de N, possui um número muito grande de soluções. Assim, favorece-se o uso de técnicas de busca local, que procuram encontrar rapidamente uma solução sem precisar explorar todo o espaço de possibilidades. No entanto, diferente dos SAT solvers, não é garantido a completude, logo, é possível encontrar uma solução rapidamente quando ela existe, mas não consegue-se provar que o problema é insolúvel.

5.4.4. Conclusão da análise

A modelagem do problema por meio de SAT tem a vantagem de reutilizar um solver genérico, que já possui técnicas de busca e otimização desenvolvidas e amplamente testadas para que, assim, não seja necessário implementar manualmente estratégias de poda, propagação e outras heurísticas, simplificando o processo de desenvolvimento.

No entanto, essa abordagem também apresenta um custo relacionado à codificação do problema. Como foi citado anteriormente, o número de cláusulas cresce de forma cúbica em N, devido principalmente às restrições de "no máximo uma" nas linhas, colunas e diagonais. Além disso, algumas cláusulas podem ser redundantes e acabam sendo geradas mesmo que um algoritmo especializado consiga evitar esse processo.

Para o N-Rainhas especificamente, algoritmos especializados como backtracking com poda ou min-conflicts podem apresentar melhor desempenho para valores muito grandes de N. Essas abordagens são desenvolvidas especificamente para explorar as características do problema e conseguem evitar parte do custo envolvido na modelagem por SAT.

Entretanto, a abordagem SAT se destaca principalmente pela sua flexibilidade pois, caso seja necessário adicionar novas restrições ou trabalhar com alguma variação do problema, basta modificar ou acrescentar as famílias de cláusulas correspondentes, sem a

necessidade de alterar toda a lógica do algoritmo de busca. Essa facilidade de adaptação torna-se uma vantagem essencial do ponto de vista de engenharia, mesmo que o SAT solver em desempenho bruto não seja sempre a opção mais rápida.